

Give an Inch and Take a Mile?

Effects of Adding Reliable Knowledge to Heuristic Feature Tracing

Sandra Greiner¹ Alexander Schultheiß³ Paul M. Bittner²
Thomas Thüm² Timo Kehrer³

¹ACP Section, University of Southern Denmark, DK

²Software Engineering Group, TU Braunschweig, D

³Software Engineering Group, University of Bern, CH

SE, February 2026, Bern



Large Evolving Highly-Configurable Software



Sign up

nasa / fprime

Notifications Fork 1.2k Star 9.3k

Code Issues 114 Pull requests 4 Discussions Actions Projects 8 Wiki

master - Go to file Code - About

LeStarch Update CONTRIBUTING.md (#1670) on Sep 26, 2022 4,770

.github	lestarch: removing permissions-intensive rep...	6 months ago
Autocoders	Update check-spelling to v0.0.20 (#1583)	7 months ago
CFDP	mstarch: all cmake is now targets, deployment...	2 years ago
Drv	Add systime.h include to lpsocket for mud c...	9 months ago
Fpp	Revise Ref topology model	2 years ago
Fw	Update check-spelling to v0.0.20 (#1583)	7 months ago
Os	Only support set_cpu_affinity with glibc (#1517)	9 months ago
RPI	Latest ActiveRateGroup updates (#1510)	9 months ago
Ref	Update/ref int tests (#1479)	9 months ago
sTest	Revise Deframer design and implementation (...)	last year
Swc	Update check-spelling to v0.0.20 (#1583)	7 months ago
Utils	Use identity comparison with singleton (#1501)	9 months ago
ci	lestarch: initial implementation of RPI integrat...	2 years ago
cmake	Update check-spelling to v0.0.20 (#1583)	7 months ago
config	Latest ActiveRateGroup updates (#1510)	9 months ago
docs	Update check-spelling to v0.0.20 (#1583)	7 months ago
.clang-tidy	Replace NULL with nullptr (#1049)	2 years ago
.dockerignore	Fix #1215 Remove incorrect references to non...	2 years ago
.gitignore	Fix #1215 Remove incorrect references to non...	2 years ago

F - A flight software and embedded systems framework

nasa.github.io/fprime

raspberrypi components
real-time framework embedded
cpp nasa embedded-systems
flight spacecraft
object-oriented programming
flight software fprime

Readme
Apache-2.0 license
9.3k stars
258 watching
1.2k forks

Releases 15

Release v2.1.1: fprime... Latest
on Aug 15, 2022

+ 14 releases

Contributors 118

Large Evolving Highly-Configurable Software



numpy

Sign up

nasa / fprime Public

Code Issues 114 Pull requests 4 Discussions 0 Actions 0 Projects 8 Wiki

master

Go to file Code - About

LeStarch Update CONTRIBUTING.md (#1670) ✓ On Sep 26, 2022 4,776

- .github lestarch: removing permissions-intensive rep... 6 months ago
- Autocoders Update check-spelling to v0.0.20 (#1583) 7 months ago
- CFDP mstarch: all cmake is now targets, deployment... 2 years ago
- Drv Add systime.h include to lpsocket for mud c... 9 months ago
- Fpp Revise Ref topology model 2 years ago
- Fw Update check-spelling to v0.0.20 (#1583) 7 months ago
- Os Only support set_cpu_affinity with glibc (#1517) 9 months ago
- RPI Latest ActiveRateGroup updates (#1510) 9 months ago
- Ref Update/ref int tests (#1479) 9 months ago
- sTest Update Deframer design and implementation (... last year
- Swc Revise check-spelling to v0.0.20 (#1583) 7 months ago
- Utils Use identity comparison with singleton (#1501) 9 months ago
- ci lestarch: initial implementation of RPI integrat... 2 years ago
- cmake Update check-spelling to v0.0.20 (#1583) 7 months ago
- config Latest ActiveRateGroup updates (#1510) 9 months ago
- docs Update check-spelling to v0.0.20 (#1583) 7 months ago
- clang-tidy Replace NULL with nullptr (#1049) 2 years ago
- dockerignore Fix #1215 Remove incorrect references to non... 2 years ago
- .gitignore Fix #1215 Remove incorrect references to non... 2 years ago

F - A flight software and embedded systems framework

nasa.github.io/fprime

- real-time framework embedded
- cpp nasa embedded-systems
- flight spaceflight
- object-oriented programming
- flight-software fprime

Readme Apache-2.0 license 9.3k stars 258 watching 1.2k forks

Releases 15

Release v2.1.1: fprime... on Aug 19, 2022

+ 14 releases

Contributors 118

Showing 13 changed files with 11 additions and 110 deletions.

Filter changed files

- .github/workflows
- linux.yml
- doc
- release/upcoming_changes
- 24717.change.rst
- source/reference
- arrays.ndarray.rst
- global_state.rst
- meson_options.txt
- numpy
- ._pytesttester.py
- core
- config.h.in
- meson.build
- numeric.py
- src/multiairray
- ctors.c
- textreading
- rows.c
- tests
- test_multiairray.py
- test_regression.py

```
4080 4087 }
4081 4088     if (not_cf_config) {
4082 4089         'objflags = (!'objflags)|NPY_ARRAY_C_CONTIGUOUS &
```

```
7 numpy/core/src/multiairray/textreading/rows.c
@@ -297,12 ~297,7 @@ read_rows(stream *,
297 297     /*
298 298         data_array = (PyObject *)PyArray_SimpleNewFromDescr(
299 299             ndim, result_shape, out_descr);
300 - #ifndef NPY_RELAXED_STRIDES_DEBUG
301 -     /* Incompatible with NPY_RELAXED_STRIDES_DEBUG due to growing */
302 -     if (result_shape[0] == 1) {
303 -         PyArray_STRIDES(data_array)[0] = row_size;
304 -     }
305 - #endif /* NPY_RELAXED_STRIDES_DEBUG */
306 +
307 +     if (data_array == NULL) {
308 308         goto error;
309 309     }
```

```
25 numpy/core/tests/test_multiairray.py
@@ -3520,8 ~3520,6 @@ def test_ravel(self):
3520 3520     # 1-element 1d strides test:
3521 3521     a = np.array([1])
3522 3522     a.strides = (123, 432)
3523 - # If the following stride is not 8, NPY_RELAXED_STRIDES_DEBUG is
3524 - # messing them up on purpose:
3525 3525     if np.ones(1).strides == (8,):
3526 3526         assert(np.share_memory(a.ravel('K'), a))
3527 3527         assert_equal(a.ravel('K').strides, (a.dtype.itemsize,))
3528 +
3529 + @pytest.mark.parametrize('record', [True, False])
3530 + def test_test_record(self, record):
3531 3531     assert_equal(y.format, 'T(bia=hb1:ic1:d:q:dx:8:0:tf=I:q:L:h:Q:
3532 3532     # Note:
3533 3533     assert_equal(y.format, 'T(bia=hb1:ic1:d:q:dx:8:0:tf=I:q:L:h:Q:
3534 3534     # Cannot test if NPY_RELAXED_STRIDES_DEBUG changes the strides
3535 -     if not (np.ones(1).strides[0] == np.info(np.intp).max):
3536 -         assert_equal(y.strides, (sz,))
3537 +     assert_equal(y.strides, (sz,))
```

Large Evolving Highly-Configurable Software



numpy

The screenshot shows the GitHub interface for the `nasa/frime` repository. The main content area lists several pull requests, with the first one, `Lefstarch Update CONTRIBUTING.md (#1670)`, highlighted. This pull request is from the user `lefstarch` and is marked as 'Merged'. The right sidebar provides additional information about the repository, including its description: 'F - A flight software and embedded systems framework', and a list of tags: `raspberry-gli`, `components`, `real-time`, `framework`, `embedded`, `c++`, `naive`, `embedded-systems`, `flight`, `spaceflight`, `object-oriented-programming`, `flight-software`, and `frime`.

Showing 13 changed files with 11 additions and 110 deletions

Filter changed files

- github/workflows
- linux.yml
- doc
- release/upcoming_changes
- 24717.change.rst
- source/reference
- arrays.ndarray.rst
- global_state.rst
- meson_options.txt
- numpy
- _pytesttester.py
- core
- config.h.in
- meson.build
- numeric.py
- src/multiarray

```

4088 4087 }
4081 4088         if (not_of_contig) {
4082 4089             'objflags = (('objflags)|NPY_ARRAY_C_CONTIGUOUS) &

```

```

v 7 7 numpy/core/src/multiarray/textreading/rows.c
+ 00 -297,12 +297,7 @@ read_rows(stream *,
297 297         /*
298 298         data_array = (PyArrayObject *)PyArray_SimpleNewFromDescr(
299 299         ndia, result_shape, out_desc);
300 - #ifdef NPY_RELAXED_STRIDES_DEBUG
301 - // Incompatible with NPY_RELAXED_STRIDES_DEBUG due to growing */
302 - if (result_shape[0] == 1) {
303 -     PyArray_STRIDES[data_array][0] = row_size;
304 - }
305 - #endif /* NPY_RELAXED_STRIDES_DEBUG */
306 +
307 301         if (data_array == NULL) {
308 302             goto error;
309 303         }

```

```

v 25 25 numpy/core/tests/test_multiarray.py
+ 00 -3520,8 +3520,6 @@ def test_ravel(self):
3520 3520         # 1-element tidy strides test:
p_array[[[1]]]
lens = (125, 432)
the following stride is not 8, NPY_RELAXED_STRIDES_DEBUG is
then set up on purpose:
ones(1).strides == (8,):
sort_(np.may_share_memory(a.ravel('K'), a))
sort_(np.may_share_memory(a.ravel('K').strides, (a.dtype.itemsize,))
830,7 @@ def test_export_record(self):
sort_equal(y.format, 'T[0:a-wh:b:c:d;q;d:B@f:I-g:L:h;q:
sort_equal(y.format, 'T[0:a-wh:b:c:d;q;d:B@f:I-g:L:h;q:
not test if NPY_RELAXED_STRIDES_DEBUG changes the strides
t (eq.ones(1).strides[0] == eq.info(np.inp).max):
sort_equal(y.strides, (sz,))
t_equal(y.strides, (sz,))

```

options realized?

change them?

cs, cost, ...)

Where are configuration options realized?

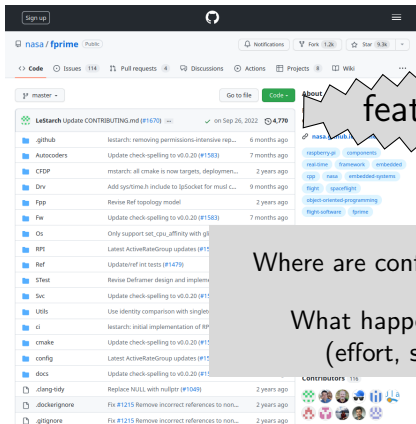
What happens if we change them?
(effort, side effects, cost, ...)

Large Evolving Highly-Configurable Software



numpy

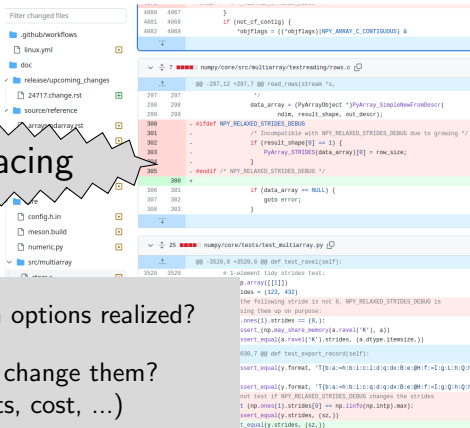
Showing 13 changed files with 11 additions and 110 deletions



feature tracing

Where are configuration options realized?

What happens if we change them?
(effort, side effects, cost, ...)



Background

How does feature tracing work?

Example: Graph Library

Example: Graph Library

superimposed configurable software

```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
        { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}  
  
public class Node{}  
  
// #IFDEF Color  
public class Color {}  
// #ENDIF
```


Example: Graph Library

superimposed configurable software

```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
        { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}  
  
public class Node{}  
  
// #IFDEF Color  
public class Color {}  
// #ENDIF
```

multiple variants

```
1 /// V1: {G, E, W}  
2 // FEAT: Graph  
3 interface Graph {  
4     List<Node> nodes();  
5     // FEAT: E  
6     List<Edge> edges();  
7     List<Node> nodes(double w);  
8 }
```

Example: Graph Library

superimposed configurable software

```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
        { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}  
  
public class Node{}  
  
// #IFDEF Color  
public class Color {}  
// #ENDIF
```

multiple variants

```
1 /// V1: {G, E, W}  
2 // FEAT: Graph  
3 interface Graph {  
4     List<Node> nodes();  
5     // FEAT: E  
6     List<Edge> edges();  
7     List<Node> nodes(double w);  
8 }
```

```
1 /// V2: {G, E, C}  
2 interface Graph {  
3     List<Node> nodes();  
4     // FEAT: C  
5     List<Node> nodes(Color c);  
6     List<Edge> edges();  
7     Graph subGraph(Color c);  
8 }
```

Example: Graph Library

superimposed configurable software

```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
        { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}  
  
public class Node{}  
  
// #IFDEF Color  
public class Color {}  
// #ENDIF
```

multiple variants

```
1 /// V1: {G, E, W}  
2 // FEAT: Graph  
3 interface Graph {  
4     List<Node> nodes();  
5     // FEAT: E  
6     List<Edge> edges();  
7     List<Node> nodes(double w);  
8 }
```

```
1 /// V2: {G, E, C}  
2 interface Graph {  
3     List<Node> nodes();  
4     // FEAT: C  
5     List<Node> nodes(Color c);  
6     List<Edge> edges();  
7     Graph subGraph(Color c);  
8 }
```

```
1 /// V3: {G, E, D}  
2 interface Graph {  
3     List<Node> nodes();  
4     List<Edge> edges();  
5     List<Edge> incomEdges(Node n);  
6     // FEAT: C  
7     Graph subGraph(Color c);  
8 }
```

Proactive Tracing



```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
    { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}  
  
public class Node{}  
  
// #IFDEF Color  
public class Color {}  
// #ENDIF
```

Proactive Tracing



through

code comments/annotations

```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
    { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}  
  
public class Node{}  
  
// #IFDEF Color  
public class Color {}  
// #ENDIF
```

Proactive Tracing



through

code comments/annotations
development on 'feature' branches

```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
    { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}  
  
public class Node{}  
  
// #IFDEF Color  
public class Color {}  
// #ENDIF
```

Proactive Tracing



through

- code comments/annotations
- development on 'feature' branches
- semi-automated feature trace recording

```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
    { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}  
  
public class Node{}  
  
// #IFDEF Color  
public class Color {}  
// #ENDIF
```

Proactive Tracing



through

- code comments/annotations
- development on 'feature' branches
- semi-automated feature trace recording
- semi-automated nudging (based on reinforcement learning)

```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
    { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}  
  
public class Node{  
  
    // #IFDEF Color  
    public class Color {}  
    // #ENDIF
```


Proactive Tracing



```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
    { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}  
  
public class Node{  
  
    // #IFDEF Color  
    public class Color {}  
    // #ENDIF
```

through

- code comments/annotations
- development on 'feature' branches
- semi-automated feature trace recording
- semi-automated nudging (based on reinforcement learning)

requires

Proactive Tracing



```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
    { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}  
  
public class Node{}  
  
// #IFDEF Color  
public class Color {}  
// #ENDIF
```

through

- code comments/annotations
- development on 'feature' branches
- semi-automated feature trace recording
- semi-automated nudging (based on reinforcement learning)

requires

- discipline
- development practices and guidelines
- remains a **manual** task

Proactive Tracing



```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
    { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}  
  
public class Node{  
  
    // #IFDEF Color  
    public class Color {}  
    // #ENDIF
```

through

- code comments/annotations
- development on 'feature' branches
- semi-automated feature trace recording
- semi-automated nudging (based on reinforcement learning)

requires

- discipline
- development practices and guidelines
- remains a **manual** task

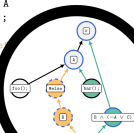
⇒ often neglected
⇒ missed opportunity

Retroactive Tracing

```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
    { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}
```

```
public class Node{}  
  
// #IFDEF Color  
public class Color {}  
// #ENDIF
```

```
#ifdef A  
foo();  
-#else  
-#ifdef  
+ bar();  
+endif  
+if B  
baz();  
-#endif  
endif
```

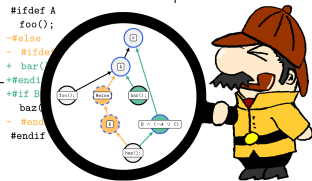


Retroactive Tracing

```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
    { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}  
  
public class Node{}  
  
// #IFDEF Color  
public class Color {}  
// #ENDIF
```

through

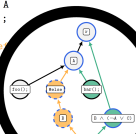
manual code inspection



Retroactive Tracing

```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
    { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}  
  
public class Node{}  
  
// #IFDEF Color  
public class Color {}  
// #ENDIF
```

```
#ifdef A  
foo();  
-#else  
- #ifdef  
+ bar();  
+ #endif  
+ #if B  
baz();  
- #endif  
#endif
```



through

manual code inspection

dynamic techniques (which compare executions)

Retroactive Tracing

```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
    { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}  
  
public class Node{}  
  
// #IFDEF Color  
public class Color {}  
// #ENDIF
```

```
#ifdef A  
foo();  
-#else  
-#ifdef  
+ bar();  
+endif  
+if B  
baz();  
-#endif  
#endif
```



through

manual code inspection

dynamic techniques (which compare executions)

static techniques (e.g., slicing, clone detection, comparison of variants)

Retroactive Tracing

```
public class Graph {

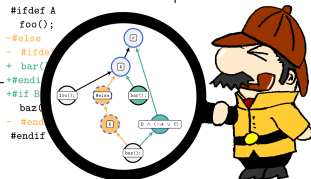
    List<Node> getNodes(Color c) { ... }
    List<Edge> getIncomEdges(Node n) { ... }

    // #IFDEF Weighted
    List<Edge> getEdges(double w)
    { ... }
    // #ENDIF

    Graph subGraph(double w) { ... }
}

public class Node{}

// #IFDEF Color
public class Color {}
// #ENDIF
```



through

manual code inspection

dynamic techniques (which compare executions)

static techniques (e.g., slicing, clone detection, comparison of variants)

combinations thereof

Retroactive Tracing

```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
    { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}  
  
public class Node{}  
  
// #IFDEF Color  
public class Color {}  
// #ENDIF
```

```
#ifdef A  
foo();  
-#else  
- #ifdef  
+ bar();  
+ #endif  
+ #if B  
baz();  
- #endif  
#endif
```



through

manual code inspection

dynamic techniques (which compare executions)

static techniques (e.g., slicing, clone detection, comparison of variants)

combinations thereof

requires

heuristics

execution effort and data

while automated, reliability?

Retroactive Tracing

```
public class Graph {  
  
    List<Node> getNodes(Color c) { ... }  
    List<Edge> getIncomEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
    { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}  
  
public class Node{}  
  
// #IFDEF Color  
public class Color {}  
// #ENDIF
```

```
#ifdef A  
foo();  
-#else  
- #ifdef  
+ bar();  
+ #endif  
+ #if B  
baz();  
- #endif  
#endif
```



through

manual code inspection

dynamic techniques (which compare executions)

static techniques (e.g., slicing, clone detection, comparison of variants)

combinations thereof

requires

heuristics

execution effort and data

while automated, reliability?

→ **manual** task

Feature Tracing - Summary

proactive

reliable*

pin-pointed

Feature Tracing - Summary

proactive

- reliable*

- pin-pointed

- without immediate benefit

- additional documentation burden

Feature Tracing - Summary

proactive

- reliable*

- pin-pointed

- without immediate benefit

- additional documentation burden

(*exclude uncertainty, feature interactions...)

Feature Tracing - Summary

proactive

reliable*

pin-pointed

without immediate benefit

additional documentation burden

(*exclude uncertainty, feature interactions...)

retroactive

based on heuristics

Feature Tracing - Summary

proactive

reliable*

pin-pointed

without immediate benefit

additional documentation burden

(*exclude uncertainty, feature interactions...)

retroactive

based on heuristics

automated: static or dynamic or hybrid

Feature Tracing - Summary

proactive

reliable*

pin-pointed

without immediate benefit

additional documentation burden

(*exclude uncertainty, feature interactions...)

retroactive

based on heuristics

automated: static or dynamic or hybrid

accuracy depends on ...

→ less reliable

Feature Tracing - Summary

proactive

reliable*

pin-pointed

without immediate benefit

additional documentation burden

(*exclude uncertainty, feature interactions...)

retroactive

based on heuristics

automated: static or dynamic or hybrid

accuracy depends on ...

→ less reliable

⇒ plethora of techniques, but not optimal

Feature Traceability

What is it and why should we care?

Feature Traceability

What is it and why should we care?

How can we do better?

How can retroactive feature tracing
benefit from proactive traces?

How can retroactive feature tracing
benefit from proactive traces?

Experiments with comparison-based feature location

Comparison-Based Feature Location

```
1 /// V1: {G, E, W}
2 // FEAT: Graph
3 interface Graph {
4     List<Node> nodes();
5     // FEAT: E
6     List<Edge> edges();
7     List<Node> nodes(double w);
8 }
```

```
1 /// V3: {G, E, D}
2 interface Graph {
3     List<Node> nodes();
4     List<Edge> edges();
5     List<Edge> incomEdges(Node n);
6     // FEAT: C
7     Graph subGraph(Color c);
8 }
```

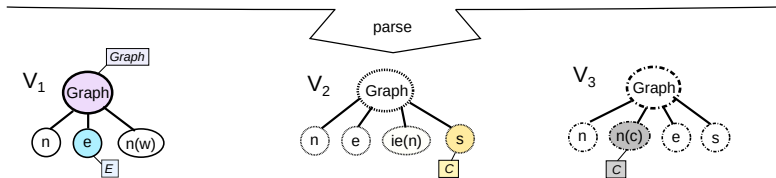
```
1 /// V2: {G, E, C}
2 interface Graph {
3     List<Node> nodes();
4     // FEAT: C
5     List<Node> nodes(Color c);
6     List<Edge> edges();
7     Graph subGraph(Color c);
8 }
```

Comparison-Based Feature Location

```
1 /// V1: {G, E, W}
2 // FEAT: Graph
3 interface Graph {
4   List<Node> nodes();
5   // FEAT: E
6   List<Edge> edges();
7   List<Node> nodes(double w);
8 }
```

```
1 /// V3: {G, E, D}
2 interface Graph {
3   List<Node> nodes();
4   List<Edge> edges();
5   List<Edge> incomEdges(Node n);
6   // FEAT: C
7   Graph subGraph(Color c);
8 }
```

```
1 /// V2: {G, E, C}
2 interface Graph {
3   List<Node> nodes();
4   // FEAT: C
5   List<Node> nodes(Color c);
6   List<Edge> edges();
7   Graph subGraph(Color c);
8 }
```



Comparison-Based Feature Location

```

1 /// V1: {G, E, W}
2 // FEAT: Graph
3 interface Graph {
4   List<Node> nodes();
5   // FEAT: E
6   List<Edge> edges();
7   List<Node> nodes(double w);
8 }

```

```

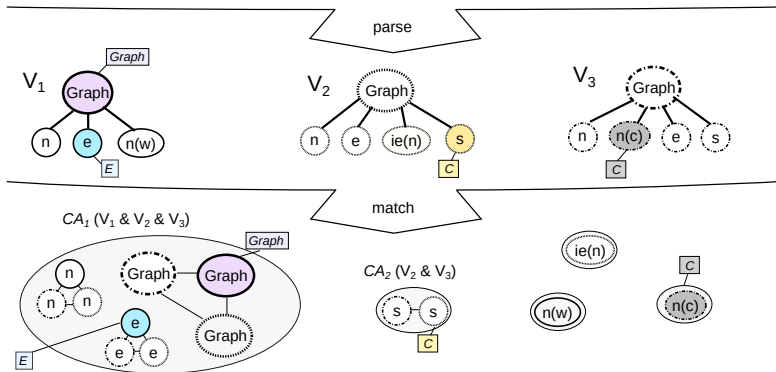
1 /// V3: {G, E, D}
2 interface Graph {
3   List<Node> nodes();
4   List<Edge> edges();
5   List<Edge> incomEdges(Node n);
6   // FEAT: C
7   Graph subGraph(Color c);
8 }

```

```

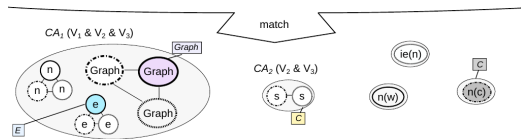
1 /// V2: {G, E, C}
2 interface Graph {
3   List<Node> nodes();
4   // FEAT: C
5   List<Node> nodes(Color c);
6   List<Edge> edges();
7   Graph subGraph(Color c);
8 }

```



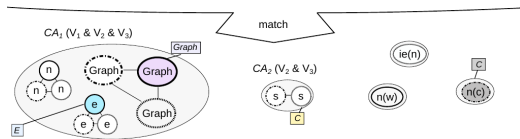
Comparison-Based Feature Location

annotation per node (based on features shared in configurations)



Comparison-Based Feature Location

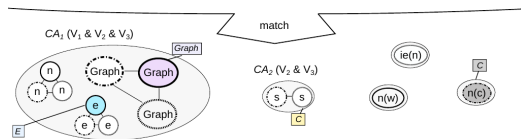
annotation per node (based on features shared in configurations)



Feature Trace

Comparison-Based Feature Location

annotation per node (based on features shared in configurations)



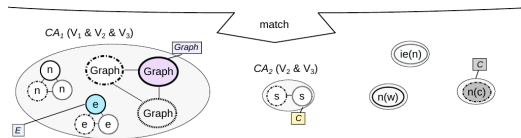
Feature Trace

<code>Graph $\wedge$ Edge</code>	<code>interface Graph, nodes(), edges(),</code>
<code>Weighted</code>	<code>nodes (double w)</code>
<code>Colored</code>	<code>nodes(Color c), subGraph()</code>
<code>Directed</code>	<code>incomingEdges(Edge e)</code>

Comparison-Based Feature Location

annotation per node (based on features shared in configurations)

Controlled Experiment



Feature Trace

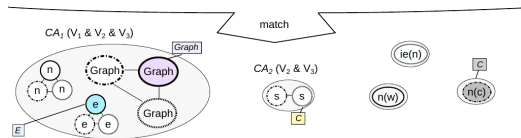
Graph $\wedge$ Edge	interface Graph, nodes(), edges(),
Weighted	nodes (double w)
Colored	nodes(Color c), subGraph()
Directed	incomingEdges(Edge e)

Comparison-Based Feature Location

annotation per node (based on features shared in configurations)

Controlled Experiment

propagate 'reliable annotations'
whenever possible (not contradicting
among one set)

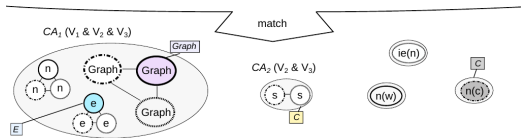


Feature Trace

Graph $\wedge$ Edge	interface Graph, nodes(), edges(),
Weighted	nodes (double w)
Colored	nodes(Color c), subGraph()
Directed	incomingEdges(Edge e)

Comparison-Based Feature Location

annotation per node (based on features shared in configurations)



Controlled Experiment

- propagate 'reliable annotations' whenever possible (not contradicting among one set)
- measure quantitatively

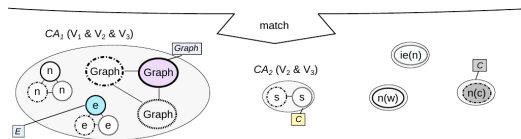
effect of adding proactive mappings
per variant

Feature Trace

Graph $\wedge$ Edge	interface Graph, nodes(), edges(),
Weighted	nodes (double w)
Colored	nodes(Color c), subGraph()
Directed	incomingEdges(Edge e)

Comparison-Based Feature Location

annotation per node (based on features shared in configurations)



Controlled Experiment

propagate 'reliable annotations'
whenever possible (not contradicting
among one set)
measure quantitatively

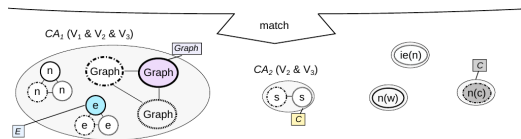
effect of adding proactive mappings
per variant
effect of number of compared
variants

Feature Trace

<code>Graph $\wedge$ Edge</code>	<code>interface Graph, nodes(), edges(),</code>
<code>Weighted</code>	<code>nodes (double w)</code>
<code>Colored</code>	<code>nodes(Color c), subGraph()</code>
<code>Directed</code>	<code>incomingEdges(Edge e)</code>

Comparison-Based Feature Location

annotation per node (based on features shared in configurations)



Feature Trace

Graph $\wedge$ Edge	interface Graph, nodes(), edges(),
Weighted	nodes (double w)
Colored	nodes(Color c), subGraph()
Directed	incomingEdges(Edge e)

Controlled Experiment

propagate 'reliable annotations'
whenever possible (not contradicting
among one set)
measure quantitatively

effect of adding proactive mappings
per variant
effect of number of compared
variants

⇒ increase in accuracy?

⇒ invest in exploiting proactive knowledge?

Subject Systems: Marlin, ArgoUML, VIM; OpenVPN, BusyBox

Subject Systems: Marlin, ArgoUML, VIM; OpenVPN, BusyBox

Groundtruth: VEVOS, Benchmark generator for highly configurable software

Subject Systems: Marlin, ArgoUML, VIM; OpenVPN, BusyBox

Groundtruth: VEVOS, Benchmark generator for highly configurable software

Measure:

Agreement between ground truth and computed mapping to keep a line of code in variant

Subject Systems: Marlin, ArgoUML, VIM; OpenVPN, BusyBox

Groundtruth: VEVOS, Benchmark generator for highly configurable software

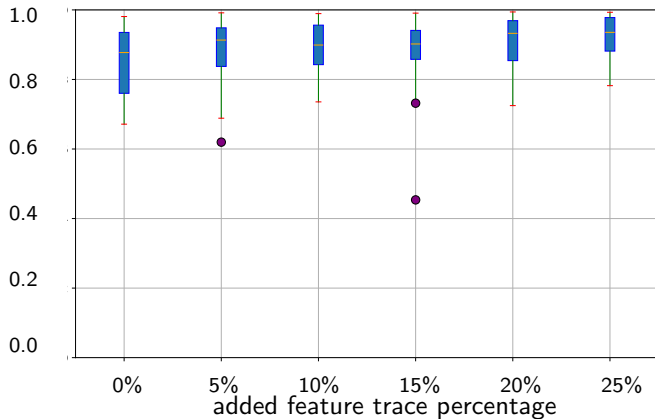
Measure:

Agreement between ground truth and computed mapping to keep a line of code in variant

Compute: Precision, Recall, F1-Score

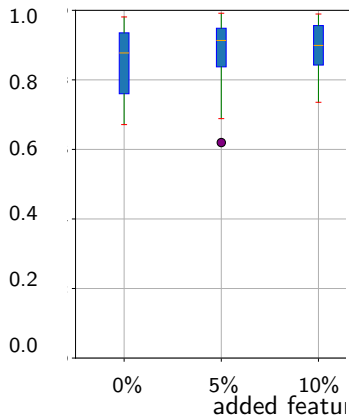
Result: Busybox (3 variants) as one example

precision

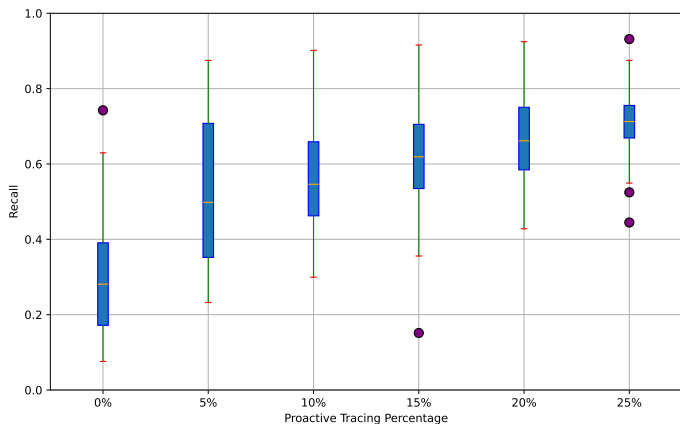


Result: Busybox (3 variants) as one example

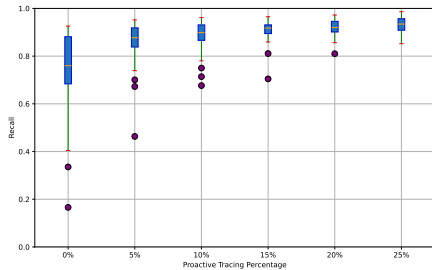
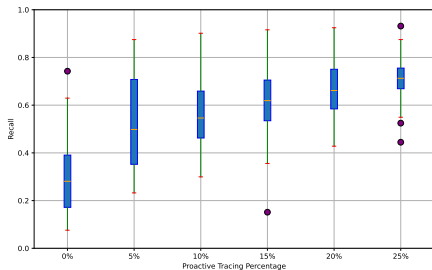
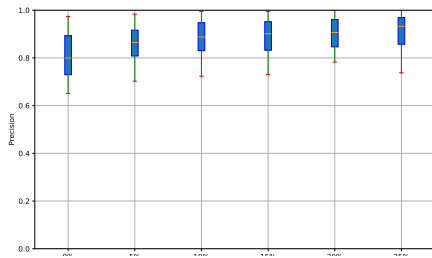
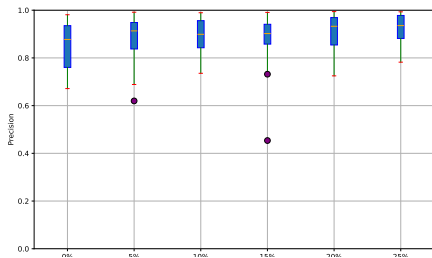
precision



recall

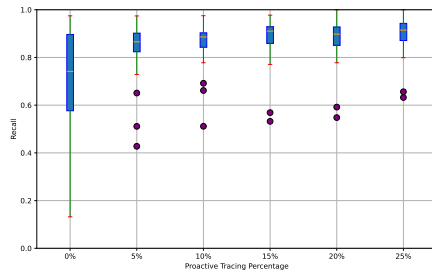
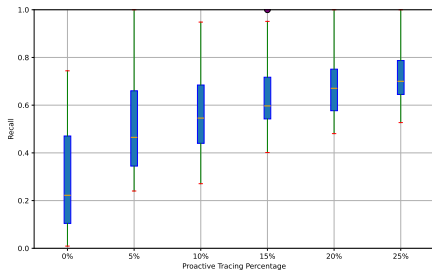
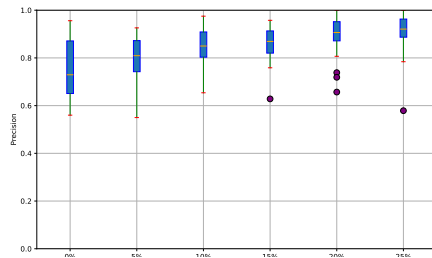
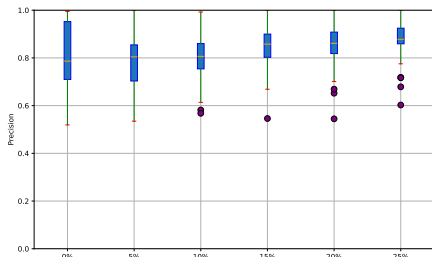


Results: Busybox (rows: precision & recall; columns: 3 & 7 variants)



Results: VIM (rows: precision & recall; columns: 3 & 7 variants)

Results: VIM (rows: precision & recall; columns: 3 & 7 variants)



only 5% of proactive traces raise the overall accuracy by 10-20%

Results: Summary

only 5% of proactive traces raise the overall accuracy by 10-20%

difference between precision and recall

- few false positives

- rate of false negatives dropped

Results: Summary

only 5% of proactive traces raise the overall accuracy by 10-20%

difference between precision and recall

- few false positives

- rate of false negatives dropped

the more variants, the higher the accuracy (with and without added traces)

- that's an effect of comparison-based technique

- but does not for all subjects (e.g., Marlin only 50% recall for one set of 7 variants)

Results: Summary

only 5% of proactive traces raise the overall accuracy by 10-20%

difference between precision and recall

- few false positives

- rate of false negatives dropped

the more variants, the higher the accuracy (with and without added traces)

- that's an effect of comparison-based technique

- but does not for all subjects (e.g., Marlin only 50% recall for one set of 7 variants)

⇒ potential to exploit proactive traces further

Results: Summary

only 5% of proactive traces raise the overall accuracy by 10-20%

difference between precision and recall

- few false positives

- rate of false negatives dropped

the more variants, the higher the accuracy (with and without added traces)

- that's an effect of comparison-based technique

- but does not for all subjects (e.g., Marlin only 50% recall for one set of 7 variants)

⇒ potential to exploit proactive traces further

⇒ examine the effect in further techniques, also qualitatively

Results: Summary

only 5% of proactive traces raise the overall accuracy by 10-20%

difference between precision and recall

- few false positives

- rate of false negatives dropped

the more variants, the higher the accuracy (with and without added traces)

- that's an effect of comparison-based technique

- but does not for all subjects (e.g., Marlin only 50% recall for one set of 7 variants)

⇒ potential to exploit proactive traces further

⇒ examine the effect in further techniques, also qualitatively

⇒ optimize retroactive techniques but also inform semi-automated tracing (e.g. for machine learning)

Give an Inch Take a Mile?

- Effects of Reliable Feature Traces

Showing 13 changed files with 11 additions and 110 deletions.

Filter changed files

📁 `.github/workflows`

linux.yml

- release/upcoming_changes

24717.change.rst

source/reference

[arrays.ndarray.rst](#)

global_state.r

meson_options.txt

numpy

 `_pytesttester.py`

cor

 [config.h.in](#)

meson.build

numeric.py

src/multiaarray

ctors.c

▼ textreading

rows.

tests

 $\square \text{ tes_multiset}(y, p)$

test_regression.py

```
4080 4067      }
4081 4068      if (not_cf_contig) {
4082 4069          *objflags = ((*objflags)|NPY_ARRAY_C_CONTIGUOUS) &
```

```

1 00 -297,12 +297,7 00 read_rows(stream %s,
297 297 */
298 298 data_array = (PyArrayObject *)PyArray_SimpleNewFromDescr(
299 299 ndim, result_shape, out_descr);
300 - #ifndef NPY_RELAXED_STRIDES_DEBUG
301 - /* Incompatible with NPY_RELAXED_STRIDES_DEBUG due to growing */
302 - if (result_shape[0] == 1) {
303 -     PyArray_STRIDES(data_array)[0] = row_size;
304 - }
305 - #endif /* NPY_RELAXED_STRIDES_DEBUG */
306 +
307 301 if (data_array == NULL) {
308 302 goto error;
309 303 }

```

```

v 25 ■■■■ numpy/tests/test_multiarray.py
+
00 -3520,8 +3520,6 @@ def test_ravel(self):
3520 3520 # 1-element tidy strides test:
3521 3521 a = np.array([[1]])
3522 3522 a.strides = (123, 432)
3523 - # If the following stride is not 8, MPY_RELAXED_STRIDES_DEBUG is
3524 - # messing them up on purpose:
3525 3523 if np.ones(1).strides == (8,):
3526 3524     assert np.share_memory(a.ravel('K'), a)
3527 3525     assert_equal(a.ravel('K').strides, [a.dtype.itemsize,])
+
+
00 -8932,9 +8930,7 @@ def test_export_record(self):
8932 8932 assert_equal(y.format, 'T(bai:hb:ic:ld:q:d:8:e:8:f:1:g:1:h:0:q:1)')
8933 8931 else:
8934 8932     assert_equal(y.format, 'T(bai:hb:ic:ld:q:d:8:e:8:f:1:g:1:h:0:q:1)')
8935 - # Cannot test if MPY_RELAXED_STRIDES_DEBUG changes the strides
8936 - # if not (np.ones(1).strides[0] == np.infosep(np.intp).max):
8937 -     assert_equal(y.strides, (sz,))
+
+
8938 + assert_equal(y.strides, (sz,))

```


Give an Inch Take a Mile?

- Effects of Reliable Feature Traces

Proactive Tracing



```
public class Graph {
    List<Node> getNodes(Color c) { ... }
    List<Edge> getIncomEdges(Node m) { ... }

    // #IFDEF Weighted
    List<Edge> getEdges(double w)
    { ... }
    // #ENDIF

    Graph subGraph(double w) { ... }
}

public class Node{}

// #IFDEF Color
```

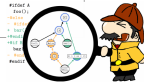
Retroactive Tracing

```
public class Graph {

    List<Node> getNodes(Color c){ ... }
    List<Edge> getIncomEdges(Node n) { ... }

    // #IFDEF Weighted
    List<Edge> getEdges(double w)
    { ... }
    // #ENDIF

    Graph subGraph(double w) { ... }
}
```



Showing 13 changed files with 11 additions and 110 deletions.

Filter changed files

📁 `.github/workflows`

linux.ytt

- doc
- release/upcoming_changes

24717.change.rst

source/reference

[arrays.ndarray.rst](#)

global state rst

```
mason_options.txt
```

YOU KNOW YOURS

[Download PDF](#)



100

[colling.it.in](#)

☐ meson.build

numeric.py

src/multibarray

ctors.c

- ▼ textreading

rows.c

tests

test_multiarray.p

test_regression.ppt

```

4880 4887
4881 4888         if (not_of_contig) {
4882 4889             'objflags = {'objflags'}\NPY_ARRAY_C_CONTIGUOUS) &

```

```

v 7 7 numpy/core/src/multiarray/textreading/rows.c
└
297 297         00 -297,12 +297,7 00 read_rows(stream %s,
297 297             */
298 298         data_array = (PyArrayObject *)PyArray_SimpleNewFromDescr(
299 299             ndim, result_shape, out_descr);
300
301 - #ifdef NPY_RELAXED_STRIDES_DEBUG
302 - /* Incompatible with NPY_RELAXED_STRIDES_DEBUG due to growing */
303 - if (result_stride[0] == 1) {
304 -     PyArray_STRIDES[data_array][0] = row_size;
305 - }
306 - #endif /* NPY_RELAXED_STRIDES_DEBUG */
307 +
308 301         if (data_array == NULL) {
309 302             goto error;
310 303         }
311
312 v 25 25 numpy/core/tests/test_multiarray.py
└
8025 8028 00 -3529,8 +3529,6 00 def test_ravel(self):
8026 8029
8027 8030     # 1-element tidy strides test:
8028 8031     a = np.array([[1]])
8029 8032     a.strides = (123, 432)
8030
8031 -     # If the following stride is not 8, NPY_RELAXED_STRIDES_DEBUG is
8032 -     # messing them up on purpose:
8033     if np.ones(1).strides == (8,):
8034         assert(np.may_share_memory(a.ravel('K'), a))
8035         assert_equal(a.ravel('K').strides, (a.dtype.itemsize,))
8036
8037 +
8038 8032 00 -8932,9 +8939,6 00 def test_export_record(self):
8039 8033
8040 8034     assert_equal(y.format, 'T(b;a=hb;ic;l;d;gds;8;e;B(f;f=i;lg;h;Q;
8041 8035         else:
8042 8036         assert_equal(y.format, 'T(b;a=hb;ic;lg;d;gds;8;e;B(f;f=i;lg;h;Q;
8043 8037         # Cannot test if NPY_RELAXED_STRIDES_DEBUG changes the strides
8044 8038         if not (np.ones(1).strides[0] == np.info(np.intp).max):
8045 8039             assert_equal(y.strides, (sz,))
8046 8040
8047 8041     assert_equal(y.strides, (sz,))

```

Give an Inch Take a Mile?

- Effects of Reliable Feature Traces

Proactive Tracing



```
public class Graph {
    List<Node> getNodes(Color c) { ... }
    List<Edge> getIncomEdges(Node m) { ... }

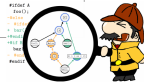
    // #IFDEF Weighted
    List<Edge> getEdges(double w)
    { ... }
    // #ENDIF

    Graph subGraph(double w) { ... }
}

public class Node{
    // #IFDEF Color
```

Retroactive Tracing

```
public class Graph {  
  
    List<Node> getNodes(Color c){ ... }  
    List<Edge> getInComEdges(Node n) { ... }  
  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
    { ... }  
    // #ENDIF  
  
    Graph subGraph(double w) { ... }  
}
```

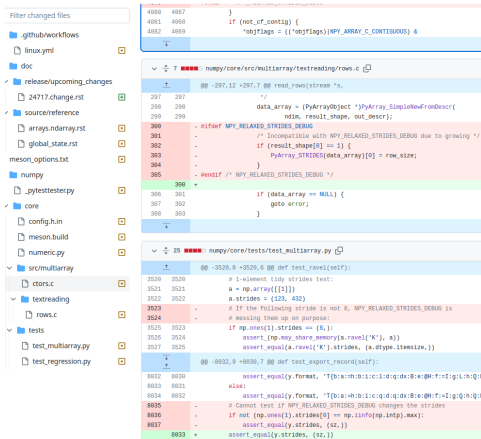


feature traceability comes with plenty of techniques

Give an Inch Take a Mile?

- Effects of Reliable Feature Traces

Showing 13 changed files with 11 additions and 110 deletions.



Proactive Tracing



```
public class Graph {
    List<Node> getNodes(Color c) { ... }
    List<Edge> getIncomEdges(Node a) { ... }

    // #IFDEF Weighted
    List<Edge> getEdges(double w)
    { ... }
    // #ENDIF

    Graph subGraph(double w) { ... }
}

public class Node{
    // #IFDEF Color
```

Retroactive Tracing

```
public class Graph {

    List<Node> getNodes(Color c){ ... }
    List<Edge> getInComEdges(Node n) { ... }

    // #IFDEF Weighted
    List<Edge> getEdges(double w)
    { ... }
    // #ENDIF

    Graph subGraph(double w) { ... }
}
```



feature traceability comes with plenty of techniques

no optimal solution, but potentials of increased accuracy with proactive traces

Give an Inch Take a Mile?

- Effects of Reliable Feature Traces

Showing 13 changed files with 11 additions and 110 deletions.

Filter changed files

📁 .github/workflows

linux.shtml

409

- release/upcoming_changes

24717.change.rst

source/reference

Downloaded from <http://ajphaphysocpharm.sagepub.com/>

□ [Grafische Darstellung](#)

- global_state.rst

meson_options.txt

numpy

 `_pytesttester.py`

core

conf

Downloaded from <http://ajph.org/>

□ **TRC30**

☐ numeric.py

src/multiarray

ctors.c

- ▼ textreading

rows.c

✓ **facts**

font-family: sans-serif; font-size: 10pt; color: #000000;">font-family: sans-serif; font-size: 10pt; color: #000000;

☐ [Log out](#)

☐ testUglyass

```

4987         if (not_of_contig) {
4988             'objflags = ({'objflags'})NPY_ARRAY_C_CONTIGUOUS &
4989         }
4990     }
4991 }
4992
4993 # numpy/core/src/multiarray/textreading/rows.c
4994
4995 @ -297,12 +297,7 @# read_rows(stream %s,
4996
4997 .t. /
4998 data_array = PyArrayObject *)PyArray_SimpleNewFromDescr(
4999 ndim, result_shape, out_descr);
5000
5001 - #ifdef NPY_RELAXED_STRIDES_DEBUG
5002 /* Incompatible with NPY_RELAXED_STRIDES_DEBUG due to growing */
5003 - if (result_shape[0] == 1) {
5004     PyArray_STRIDES(data_array)[0] = row_size;
5005 }
5006 - #endif /* NPY_RELAXED_STRIDES_DEBUG */
5007
5008 +
5009 if (data_array == NULL) {
5010     goto error;
5011 }
5012
5013 }
5014
5015 #
5016
5017 # numpy/core/tests/test_multiarray.py
5018
5019 @ -3520,8 +3520,6 @# def test_ravel(self):
5020
5021 # 1-element tidy strides test:
5022 a = np.array([[1]])
5023 a.strides = (123, 432)
5024
5025 - # If the following stride is not 8, NPY_RELAXED_STRIDES_DEBUG is
5026 - # messing them up on purpose:
5027 if np.ones(1).strides == (8,),
5028 assert(np.may_share_memory(a.ravel('K'), a))
5029 assert_equal(a.ravel('K').strides, (a.dtype.itemsize,))
5030
5031 +
5032 @ -8832,9 +8830,7 @# def test_export_record(self):
5033
5034 assert_equal(y.format, "T(b;a;b:b;i;c;d;q:d;x:B;s@M:f=I:g;l:h;q)");
5035 else:
5036 assert_equal(y.format, "T(b;a;b;b;i;c;q;d;q;d;x:B(f=f:I=g;h;q;q)");
5037
5038 - # Cannot test if NPY_RELAXED_STRIDES_DEBUG changes the strides
5039 - if not (np.ones(1).strides[0] == np.iinfo(np.intp).max):
5040     assert_equal(y.strides, (sz.,))
5041
5042 +
5043 assert_equal(y.strides, (sz.,))

```

Proactive Tracing



```
public class Graph {
    List<Node> getNodes(Color c) { ... }
    List<Edge> getIncomEdges(Node a) { ... }

    // #IFDEF Weighted
    List<Edge> getEdges(double w)
    { ... }
    // #ENDIF

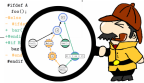
    Graph subGraph(double w) { ... }
}

public class Node{
    // #IFDEF Color
```

Retroactive Tracing

```
public class Graph {  
    List<Node> getNodes(Color c){ ... }  
    List<Edge> getInComEdges(Node n) { ... }  
    // #IFDEF Weighted  
    List<Edge> getEdges(double w)  
    { ... }  
    // #ENDIF  
    Graph subGraph(double w) { ... }  
}
```

```
public class Node{  
    // #IFDEF Color  
    public class Color {  
    // #ENDIF
```



feature traceability comes with plenty of techniques

no optimal solution, but potentials of increased accuracy with proactive traces

e.g., useful in machine learning